

Q1. Student Grade Book Application

CODE

```
n=int(input())
students={}
for _ in range(n):
    name=input()
    marks=list(map(int,input().split()))
    students[name]=marks
results={}
for name, marks in students.items():
    total=sum(marks)
    avg=total/5
    if avg ≥ 90:
        grade="O"
    elif avg ≥ 80:
        grade="A+"
    elif avg ≥ 70:
        grade="A"
    elif avg ≥ 60:
        grade="B+"
    elif avg ≥ 50:
        grade="B"
    else:
        grade="F"
    results[name]=[total, avg, grade]
topper=max(results, key=lambda x: results[x][1])
subject_means=[
    sum(marks[i] for marks in students.values())/n
    for i in range(5)
]
highest_subject=subject_means.index(max(subject_means))+1
print("name total average grade")
for name in sorted(results, key=lambda x: results[x][1], reverse=True):
    print(name, results[name][0], results[name][1], results[name][2])
print("class topper:", topper)
print("highest mean subject:", highest_subject)
```

INPUT

```
5
ravi
90 85 88 92 95
priya
80 75 82 90 88
arun
70 65 72 68 75
divya
95 98 97 96 99
kiran
55 60 58 62 57
```

OUTPUT

```
(no output)
```

Q2. Debugging and Rewriting

CODE

```
accounts={}
def create_account(acc_no, name, balance):
    accounts[acc_no]={'name': name, 'balance': balance, 'txns':[]}
def deposit(acc_no, amount):
    if acc_no in accounts:
        accounts[acc_no]['balance']+=amount
        accounts[acc_no]['txns'].append(('deposit', amount))
def withdraw(acc_no, amount):
    if accounts[acc_no]['balance'] >= amount:
        accounts[acc_no]['balance']-=amount
        accounts[acc_no]['txns'].append(('withdraw', amount))
    else:
        print("insufficient balance")
def mini_statement(acc_no):
    for t in accounts[acc_no]['txns'][-5:]:
        print(t)
create_account(1001, 'ravi', 10000)
deposit(1001, 5000)
withdraw(1001, 20000)
withdraw(1001, 3000)
mini_statement(1001)
print('balance:', accounts[1001]['balance'])
```

OUTPUT

(no output)